Architecture Case Study
GuidedAdoption Backend - Multi-Agent, Event-Driven AI Platform

Prepared for backend and platform engineering interviews (Agentic AI / LLM orchestration)

Last updated: 2026-02-23

Executive Summary

GuidedAdoption is an AI strategy SaaS platform that collects a company's context, orchestrates a multi-step analysis workflow, and produces structured artifacts (reports, PDFs, guidance documents) for leadership teams. The backend is designed as an event-driven, serverless system that runs agent-style tasks asynchronously, isolates failures with queues and DLQs, and keeps a durable audit trail across DynamoDB and S3.

- Architecture style: C4 container-level design using AWS Lambda, API Gateway, SQS, SNS, DynamoDB, S3, Redis (ElastiCache), Auth0, Stripe, and OpenAI API.
- Core pattern: request/response for user-facing endpoints plus asynchronous job execution for long-running agent workflows.
- Reliability: idempotent processing, explicit DLQs, bounded retries, and rate limiting to protect downstream systems (LLM APIs).
- Outcome: production-ready system that demonstrates modern backend architecture, distributed workflow design, and pragmatic LLM integration.

Problem Statement

Small and mid-sized businesses want AI guidance that is specific to their workflows, but they do not want to pay for long consulting engagements. The platform must turn a guided intake plus optional documents into an actionable plan, while keeping latency reasonable for the user experience and costs predictable for the business.

Design Goals

- Fast user interactions for sign-up, login, purchases, and job status checks.
- Asynchronous execution for multi-step agent workflows (minutes, not seconds).
- Durable state and auditability (who ran what, when, with what inputs, and what outputs).
- Failure isolation and graceful recovery (retries, DLQs, and manual replay).
- Cost efficiency under bursty workloads with minimal operational overhead.
- Security for identity, payments, and model access while protecting customer data.

Key Constraints

- LLM calls are slow and rate limited; they must be bounded and retried carefully.
- Users expect progress visibility while jobs run asynchronously.
- Stripe webhooks are at-least-once; purchase and entitlement logic must be idempotent.
- Serverless cold starts and concurrency limits must be managed for predictable throughput.

System Diagram

Container-level architecture (C4). This is the high-level map of services, queues, state stores, and external dependencies.

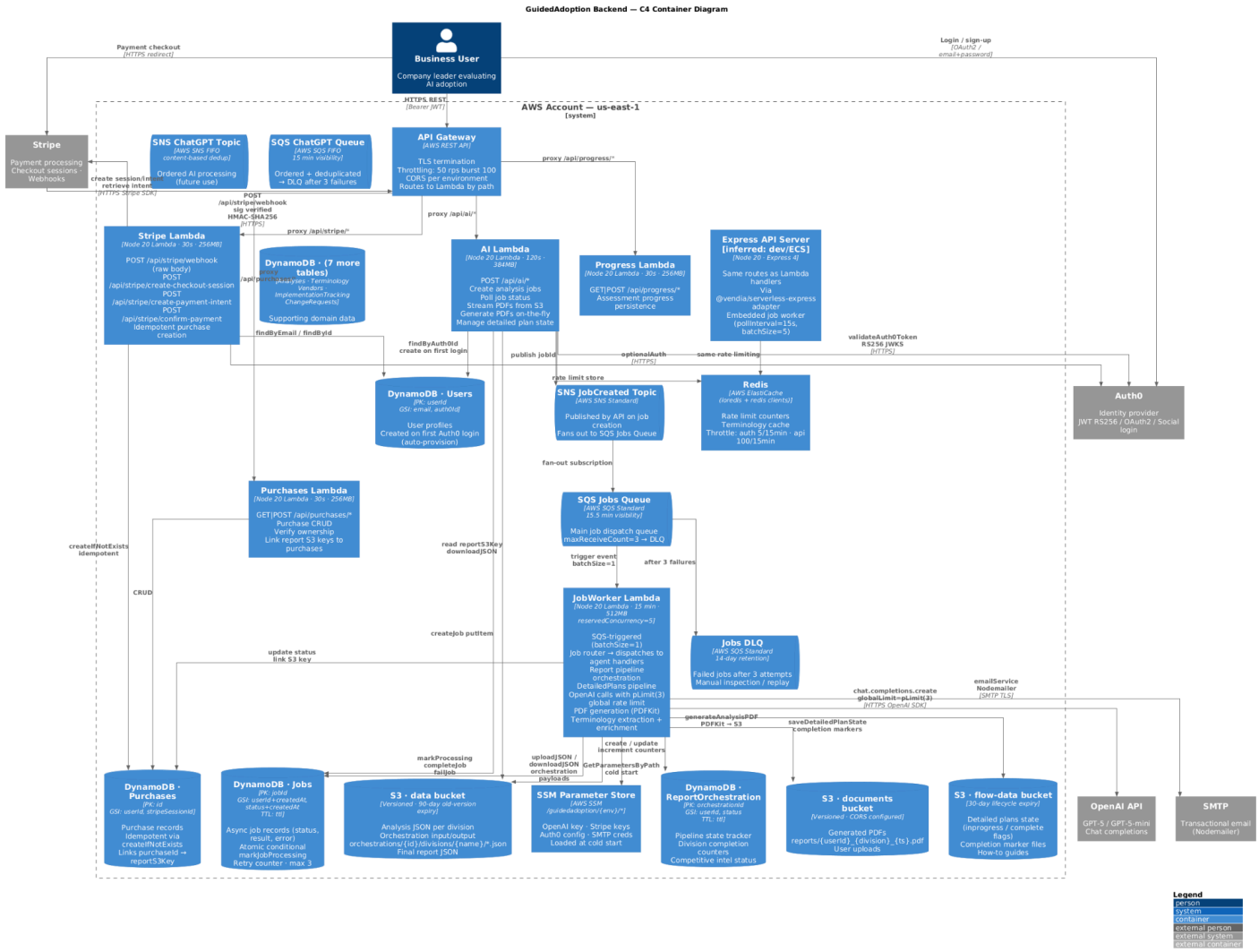

Figure 1 - GuidedAdoption Backend C4 Container Diagram

Major Components

The system is split into user-facing synchronous APIs and background asynchronous processing components:

Component	Responsibility
API Gateway	Public HTTPS entry point. Routes /api/* endpoints to Lambdas. Provides TLS termination and request throttling.
Express API Server (dev/ECS)	Dev-friendly API layer that mirrors Lambda handlers and enables embedded job workers for local iteration.
Auth0	Identity provider (JWT). Handles login/sign-up and token validation.
Stripe + Stripe Lambda	Checkout sessions and webhook ingestion. Produces durable purchase records and entitlements.
AI Lambda	Creates and tracks analysis jobs; initiates the agent workflow; persists state transitions.
SNS JobCreated Topic	Fan-out when a new job is created. Enables decoupled downstream processing.
SQS Jobs Queue + DLQ	Main dispatch queue for background work with bounded retries and dead-lettering.
JobWorker Lambda	Executes the multi-step agent workflow. Calls OpenAI, reads/writes S3 artifacts, updates DynamoDB job state.
Redis (ElastiCache)	Rate limiting and counters (per-user or per-endpoint) to protect API and LLM usage.
DynamoDB (Users, Purchases, Jobs, Orchestration)	Durable state: profiles, entitlements, job status, step tracking, pipeline state.
S3 buckets (job data, documents, flow data)	Immutable artifacts: inputs, intermediate outputs, final reports/PDFs, and completion markers.
SSM Parameter Store	Configuration and secrets references (for example OpenAI key, SMTP creds) loaded at cold start.
SMTP Email	Transactional notifications (for example purchase confirmation, job completion).

OpenAI API	LLM calls used as part of the agent workflow to generate structured outputs.

End-to-End Flows

Below are the key user and system flows. These are written the way I would explain them in a system design interview.

1) Identity and User Provisioning

1. User signs up or logs in via Auth0.
2. Backend validates JWT on API requests (token signature and audience).
3. On first authenticated request, the Users table is auto-provisioned (email and auth0Id as keys).
4. User profile becomes the anchor for purchases, jobs, and rate limiting.

2) Payment and Entitlements

5. User initiates checkout; backend creates a Stripe checkout session.
6. Stripe sends webhooks (at-least-once). Stripe Lambda verifies signature and processes events.
7. Purchases are stored in DynamoDB with an idempotency key (stripeSessionId) to prevent double writes.
8. Entitlements are linked to the user and enforced by API endpoints.

3) Job Creation and Fan-Out

9. User submits /api/ai/* request to start an analysis.
10. AI Lambda creates a Job record (status = created) and initializes orchestration state (step plan and counters).
11. SNS JobCreated event is published; subscribers fan-out into SQS Jobs Queue.
12. User polls /api/progress/* while background execution runs.

4) Agent Workflow Execution

13. JobWorker Lambda consumes SQS messages and executes steps (batch size = 1 for deterministic step ordering).
14. Worker reads inputs and intermediate artifacts from S3; writes outputs back to S3 as immutable objects.
15. Worker updates DynamoDB orchestration state (current step, progress %, completion markers).
16. Worker calls OpenAI API for specific steps only; output is constrained and validated before persistence.
17. On transient failure, message is retried up to maxReceiveCount; after that, it goes to DLQ for manual replay.

Reliability and Correctness

This system is designed for at-least-once delivery and uses defensive engineering to preserve correctness under retries.

Idempotency: Stripe and SQS both deliver at-least-once. Key write paths use idempotency keys and conditional writes to prevent duplicate purchases or job transitions.

DLQ strategy: Hard failures land in Jobs DLQ after bounded retries. This enables inspection, alerting, and controlled replay without blocking the main queue.

Progress persistence: Job progress is stored in DynamoDB so the UI can poll without coupling to worker execution.

Rate limiting: Redis counters throttle API and LLM usage, preventing runaway spend and protecting downstream dependencies.

Backpressure: Queue-based execution absorbs bursts. Lambda reserved concurrency can be tuned to cap spend and control throughput.

Security and Data Handling

- Auth0 JWT validation for API access; least-privilege IAM roles per Lambda.
- Stripe webhook signature verification (HMAC) to prevent forged events.
- Secrets and config stored in SSM Parameter Store, loaded at cold start.
- S3 buckets used as immutable artifact stores; access mediated through backend roles, not public ACLs.
- Model calls constrained to necessary inputs; outputs stored as structured artifacts with auditability.

Observability and Operations

- Structured logging per request and per job step (jobId, userId, stepName, duration).
- Metric candidates: queue depth, DLQ count, job duration distribution, Lambda errors/timeouts, LLM spend per job.
- Alerting candidates: DLQ messages > 0, sustained elevated error rate, or anomalous LLM latency.
- Operational playbook: re-drive DLQ messages after root cause fix; preserve artifacts in S3 for postmortems.

Key Tradeoffs

Why SQS + SNS instead of Step Functions?

- Queues give fine-grained backpressure control, simpler replay semantics, and lower per-transition cost. Step Functions can improve visualization but adds state machine overhead and can increase costs for high-step workflows.

Why store artifacts in S3?

- Artifacts can be large and immutable. S3 is cheap and durable. DynamoDB stores only pointers and state, keeping hot tables small and fast.

Why poll for progress?

- Polling keeps the client simple and avoids maintaining websocket infrastructure. If needed, push notifications could be added via WebSocket API or SNS notifications.

Why Redis for rate limiting?

- Fast counters and TTLs make it ideal for throttling without inflating DynamoDB writes.

Portfolio Links

Add these when publishing the public case study:

- GitHub repo: I will share temporary read only access upon request
- Short walkthrough video: https://www.youtube.com/watch?v=qRSNt1bNogs&t=12s
- Redacted sample output: Attached in separate pdf